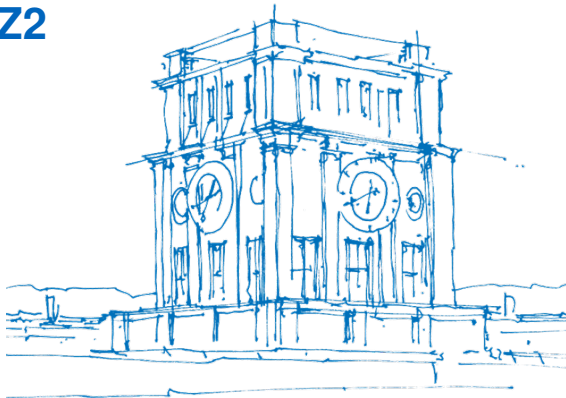


RISC-V Softcore on PYNQ-Z2

Hardware-in-the-Loop Verification

Xinran Shen Ruilai Ma

Introduction to Hardware Design Languages
and Tools



Project Objective

Goal:

- Implement a 5-stage pipelined RISC-V processor (RV32I subset)
- Deploy the processor on the PYNQ-Z2 FPGA
- Build a Hardware-in-the-Loop (HIL) verification system
- Use ARM Processing System (PS) as the host testbench

Difficulties:

- Behavioral simulation hides real hardware timing issues
- FPGA deployment exposes integration and timing challenges

Design Requirements

According to the project specification:

- Pipelined RV32I softcore
- Sorting program for 32 signed integers
- AXI-based PS–PL communication
- DONE magic number written to memory
- Jupyter-based automated verification

Success criterion:

- PS loads program and data
- Core executes correctly
- Sorted array verified automatically

System Architecture

Main Components

- ZYNQ Processing System (PS)
- AXI SmartConnect
- AXI BRAM Controller
- AXI GPIO (reset control)
- Custom RISC-V IP (PL)

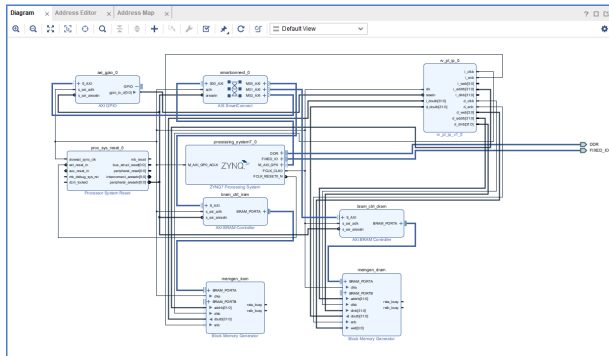


Figure: PS-PL Block Design

RISC-V Pipeline

5-stage pipeline:

- IF — Instruction Fetch
- ID — Decode
- EX — Execute
- MEM — Memory
- WB — Writeback

Supported RV32I subset:

- R-type ALU operations
- I-type ALU operations
- LW / SW
- BEQ, JAL, LUI

Hazard Handling

Data hazards

- Forwarding unit from MEM/WB to EX

Load-use hazard

- Hazard unit detects dependency
- PC freeze
- IF/ID stall
- ID/EX flush (bubble insertion)

Key challenge:

- BRAM synchronous read timing

Hardware-in-the-Loop Flow

Phase 1: Initialization

- PS loads bitstream
- Inject instruction hex
- Generate random data array
- Reset DONE flag

Phase 2: Execution

- GPIO releases reset
- RISC-V core runs autonomously

Phase 3: Verification

- PS polls DONE flag (0xCAFEBAE)
- Read sorted array
- Compare with golden result

Debugging Challenges

Issue 1: BRAM clock routing

- Vivado block automation miswired clocks
- Data arrived one cycle late
- Core executed invalid instructions

Solution

- Manual clock routing
- Use inverted BRAM clock from IP

Issue 2: Unsupported instructions

- Sorting code used `bne`
- Not supported in our ISA subset
- Rewrote program using `beq` and `jal`

Table 1: Hierarchical Resource Utilization

Module	Logic LUTs	Registers (FFs)	BRAM
Top Level (rv_pl_ip)	6,394	6,689	4
– PC0 (pc_reg)	1	32	0
– PLR1 (plr1_if_id)	43	95	0
– RF (your_reg_file_mod)	577	992	0
– PLR2 (plr2_id_ex)	324	192	0
– ALU0 (alu)	0	0	0
– HZ0 (hazard_unit)	0	1	0
– PLR3 (plr3_ex_ma)	405	137	0
– PLR4 (plr4_ma_wb)	69	136	0

Hierarchical Resource Utilization (Vivado)

Key Takeaways

- Register File and pipeline registers consume most logic resources
- Vivado heavily flattened ALU and hazard logic during synthesis
- Half-cycle critical path introduced by inverted BRAM clock
- Design comfortably fits within PYNQ-Z2 resource budget

Bonus: Single-Cycle vs. Pipelined (Summary)

Metric	Single-Cycle	Pipelined	Notes
LUTs	874	6394	logic depth / muxing
FFs	32	6689	pipeline regs add FFs
F_{max}	38.73 MHz	36.16 MHz	timing critical path

Takeaway: Pipelining should have a higher F_{max} , but the WNS is worse and the number of LUTs are significantly higher. This indicates inefficiencies in the pipelined core, which increases overhead (and Vivado is unable to compensate for it), resulting in a lower F_{max} than the supposedly least efficient single cycle version.

Bonus: Trade-off Discussion

Why pipelined should be faster

- Shorter combinational path per stage
- Higher achievable F_{max} (timing closure easier, in theory)
- Better throughput for long programs

Why single-cycle can be simpler

- No pipeline registers / forwarding / hazard control
- Often fewer FFs, but may need more muxing
- Easiest to implement and therefore cannot make as many inefficient decisions

Verification

Expected output:

- Success message
- Correctly sorted 32 integers

Observation

- Hardware completed execution
- PS successfully verified results

```
Releasing reset... RISC-V core is running!  
Polling Status Flag at 0x2000...  
Hardware finished! Flag read: 0xcafebabe  
Retrieving sorted array...  
SUCCESS: Match at index 0. Expected: -815, Got: -815  
SUCCESS: Match at index 1. Expected: -807, Got: -807  
SUCCESS: Match at index 2. Expected: -776, Got: -776  
SUCCESS: Match at index 3. Expected: -684, Got: -684  
SUCCESS: Match at index 4. Expected: -525, Got: -525  
SUCCESS: Match at index 5. Expected: -520, Got: -520  
SUCCESS: Match at index 6. Expected: -377, Got: -377  
SUCCESS: Match at index 7. Expected: -300, Got: -300  
SUCCESS: Match at index 8. Expected: -296, Got: -296  
SUCCESS: Match at index 9. Expected: -252, Got: -252  
SUCCESS: Match at index 10. Expected: -179, Got: -179  
SUCCESS: Match at index 11. Expected: -133, Got: -133  
SUCCESS: Match at index 12. Expected: -83, Got: -83  
SUCCESS: Match at index 13. Expected: -66, Got: -66  
SUCCESS: Match at index 14. Expected: 7, Got: 7  
SUCCESS: Match at index 15. Expected: 45, Got: 45  
SUCCESS: Match at index 16. Expected: 114, Got: 114  
SUCCESS: Match at index 17. Expected: 130, Got: 130  
SUCCESS: Match at index 18. Expected: 235, Got: 235  
SUCCESS: Match at index 19. Expected: 255, Got: 255  
SUCCESS: Match at index 20. Expected: 381, Got: 381  
SUCCESS: Match at index 21. Expected: 389, Got: 389  
SUCCESS: Match at index 22. Expected: 400, Got: 400  
SUCCESS: Match at index 23. Expected: 548, Got: 548  
SUCCESS: Match at index 24. Expected: 676, Got: 676  
SUCCESS: Match at index 25. Expected: 691, Got: 691  
SUCCESS: Match at index 26. Expected: 781, Got: 781  
SUCCESS: Match at index 27. Expected: 870, Got: 870  
SUCCESS: Match at index 28. Expected: 880, Got: 880  
SUCCESS: Match at index 29. Expected: 899, Got: 899  
SUCCESS: Match at index 30. Expected: 978, Got: 978  
SUCCESS: Match at index 31. Expected: 991, Got: 991
```

Success!

Conclusion

Achievements

- Functional 5-stage RISC-V on FPGA
- Successful PS–PL hardware integration
- Fully automated HIL verification

Key lessons

- Difference between simulation and real hardware behavior
- Clocking and memory timing are critical
- System-level debugging is essential